



Felvételi, 2019. július
– Alapképzés, informatika vizsga –

Hivatalból 1 pont jár. Munkaidő 3 óra.

Megjegyzések:

- A pszeudokód algoritmusban
 - az „=” értékadást jelent
 - Az $x[i][j]$ hivatkozás az x tömb i -edik sorának j -edik elemét jelenti.
 - A „minden $i = 1, n, 1$ végezd” jelentése: minden $i = 1, 2, 3, \dots, n$ értékre végezd el...
 - A „minden $i = n, 1, -1$ végezd” jelentése: minden $i = n, n-1, n-2, \dots, 1$ értékre végezd el...
- Minden egész érték tárolható integer/int típusú változóknban.
- A bemeneti adatok helyesnek tekinthetők (például, ha a feladat szövege azt mondja, hogy $1 < n < 1001$, akkor ez megelőlegezhető, és nem kell leellenőrizni).
- A programokat megjegyzésekkel kell ellátni!
- Törekedjünk hatékony megoldásokra! (maximális pontszám feltétele)

1. (1 pont) Mi lesz az $a[1..5]$ és $b[1..5]$ ötelemű tömbök tartalma az alábbi pszeudokód algoritmusrészlet nyomán, ha előzőleg az a tömb a 3, 2, 1, 5, 4, a b pedig a 6, 1, 0, 2, 9 értékeket tárolta?

```
i = 1
j = 5
amíg i < 6 ÉS j > 0 végezd
|   ha a[i] > b[j] akkor
|   |   a[i] = a[i] + 1
|   |
|   |   ha a[i] < b[j] akkor
|   |   |   b[j] = b[j] - 1
|   |
|   i = i + 1
|   j = j - 1
|
a: 3 2 2 6 4
b: 5 1 0 2 8
```

2. (1 pont) Mi lesz az $a[1..3][1..3]$ kétdimenziós tömb tartalma az alábbi pszeudokód algoritmusrészlet nyomán, ha előzőleg minden eleme 0-t tárolt?

```
minden i = 1, 2, 1 végezd
|   a[1][i] = a[1][i] + 1
|   a[i][1] = a[i][1] - 1
|
minden i = 1, 2, 1 végezd
|   a[i][3] = a[i][3] + 1
|   a[3][i] = a[3][i] - 1
|
minden i = 3, 2, -1 végezd
|   a[3][i] = a[3][i] + 1
|   a[i][3] = a[i][3] - 1
|
```

```

minden i = 3,2,-1 végezd
|   a[i][1] = a[i][1] + 1
|   a[1][i] = a[1][i] - 1
|   ■

```

```

0 0 0
0 0 0
0 0 0

```

3. (1 pont) Írjunk Pascal vagy C/C++ programot, amely beolvassa a billentyűzetről az n , m és p természetes számokat ($1 < n < 1001$; $m < n$; $p < n$), majd egy n elemű, egész értékeket tartalmazó számsorozatot, és kiírja a képernyőre (két egymás alatti sorba) a számsorozat első m (balról jobbra sorrendben), valamint utolsó p elemét (jobbról balra sorrendben).

PÉLDA

```

Input: 10 5 7
       1 3 5 7 99 8 6 4 -2 0
Output:
       1 3 5 7 99
       0 -2 4 6 8 99 7

```

```

#include <stdio.h>
int main() {
    int i, n, m, p, a[1000];
    scanf("%i%i%i", &n, &m, &p);
    for ( i = 0 ; i < n ; ++i ) {
        scanf("%i", &a[i]);
    }
    for ( i = 0 ; i < m ; ++i ) {
        printf("%i ", a[i]);
    }
    printf("\n");
    for ( i = n-1 ; i > n-p-1 ; --i ) {
        printf("%i ", a[i]);
    }
    return 0;
}

```

4. (1 pont) Írjunk Pascal vagy C/C++ programot, amely beolvassa a billentyűzetről az n értéket ($1 < n < 11$), majd megjelenít a képernyőn egy olyan $n \times n$ méretű mátrixot, amely a főátló felett 9-eseket, a főátló alatt 6-osokat, a főátlón pedig 0-ásokat tartalmaz.

PÉLDA

```

Input: 3
Output:
       0 9 9
       6 0 9
       6 6 0

```

```

#include <stdio.h>
int main() {
    int i, j, n;
    scanf("%i", &n);
    for ( i = 0 ; i < n ; ++i ) {
        for ( j = 0 ; j < n ; ++j ) {
            if ( i < j ) { printf("9 "); }
            else if ( i > j ) { printf("6 "); }
            else { printf("0 "); }
        }
    }
}

```

```

        printf("\n");
    }
    return 0;
}

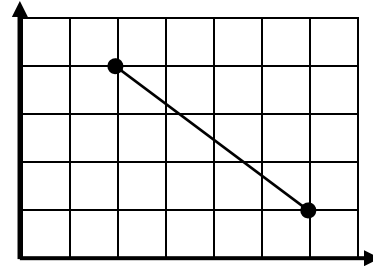
```

5. (1 pont) Írjunk Pascal vagy C/C++ függvényt, amely megkapja két síkbeli pont koordinátáit (x_1, y_1, x_2, y_2 : valós számok) és visszatéríti a pontok közti távolságot. Töltsd ki a hiányzó részt (a négyzetgyökvonáshoz felhasználhatod az `sqrt` függvényt)!

A mellékelt ábrának megfelelő paraméterértékekre

($x_1=2.00, y_1=4.00, x_2=6.00, y_2=1.00$),

a függvény az 5.00 értéket téríti vissza.



```

double Tavolsag ( double x1, double y1, double x2, double y2 ) {
    return sqrt( (x1-x2) * (x1-x2) + (y1-y2) * (y1-y2) );
}

```

```

function Tavolsag (x1:real; y1:real; x2:real; y2:real): real;
begin
    ...
end;

```

6. (1 pont) Írjunk Pascal vagy C/C++ függvényt, amely megkapja n síkbeli pont koordinátáit ($(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$), és visszatéríti, hogy egy kitüntetett pont (egyik az n közül) melyik ponthoz (a többi $n-1$ közül) van legközelebb. A függvény paraméterekként kapja meg az n értékét ($1 < n < 1001$), a pontok x és y koordinátáit tároló tömböket (legyenek $X[]$ és $Y[]$), illetve a kitüntetett pont sorszámát (legyen p), és visszatéríti a legközelebbi pont sorszámát (ha több legközelebbi pont is létezne, akkor valamelyik sorszámát térítse vissza). Töltsd ki a hiányzó részt (felhasználhatod a 5-ös feladat `Tavolsag` függvényét, amennyiben céljaidnak megfelel)!

```

int LegkozelebbiPont ( int n, double X[], double Y[], int p ) {
    --p;
    int MinPontIndex;
    double KurrensTavolsag, MinTavolsag;
    if ( p > 0 ) {
        MinTavolsag = Tavolsag( X[0], Y[0], X[p], Y[p] );
        MinPontIndex = 0;
    }
    else {
        MinTavolsag = Tavolsag( X[1], Y[1], X[p], Y[p] );
        MinPontIndex = 1;
    }
    for ( int i = 0 ; i < n ; ++i ) {
        if ( i == p ) { continue; }
        KurrensTavolsag = Tavolsag( X[i], Y[i], X[n-1], Y[n-1] );
        if ( KurrensTavolsag < MinTavolsag ) {
            MinTavolsag = KurrensTavolsag;
            MinPontIndex = i;
        }
    }
    return MinPontIndex + 1;
}

```

```
}
```

```
type TOMB = array[1..1000] of real;  
function LegkozelebbiPont (n:integer; X:TOMB; Y:TOMB; p:integer) :integer;  
begin  
    ...  
end;
```

7. (1 pont) Írjunk Pascal vagy C/C++ függvényt, amely megkapja n síkbeli pont koordinátáit $((x_1, y_1), (x_2, y_2), \dots, (x_n, y_n))$, valamint a kezdőpontnak számító pont sorszámát. A kezdőpontból a legközelebbi pontba lépünk tovább (egyenes mentén), majd innen újra a legközelebbi pontba (amelyet még nem érintettünk), és így tovább (ha egy adott pillanatban több legközelebbi pont is létezne, akkor valamelyiket válasszuk). Az utolsó pontból, ahova ily módon érünk el, visszatérünk a kezdőpontba. A függvény, paramétereként, az n értékét ($1 < n < 1001$), a pontok x és y koordinátáit tároló tömböket (legyenek $X[]$ és $Y[]$), valamint a kezdőpont sorszámát (legyen p) kapja meg. A függvény térítse vissza az így bejárt körút hosszát. Töltsd ki a hiányzó részt (felhasználhatod az 6-os feladat LegkozelebbiPont függvényét, amennyiben céljaidnak megfelel!)

```
double KorutHossz ( int n, double X[], double Y[], int p ) {  
    double temp, s = 0;  
    int m = n;  
    while ( n-1 ) {  
        temp = X[p-1]; X[p-1] = X[n-1]; X[n-1] = temp;  
        temp = Y[p-1]; Y[p-1] = Y[n-1]; Y[n-1] = temp;  
        p = LegkozelebbiPont (n, X, Y, n);  
        s += Tavolsag( X[p-1], Y[p-1], X[n-1], Y[n-1] );  
        --n;  
    }  
    s += Tavolsag( X[p-1], Y[p-1], X[m-1], Y[m-1] );  
    return s;  
}
```

```
type TOMB = array[1..1000] of real;  
function KorutHossz (n:integer; X:TOMB; Y:TOMB; p:integer): double;  
begin  
    ...  
end;
```

8. (1 pont) Adott egy n elemű bináris sorozat (elemei nullák vagy egyesek). Az első és utolsó elemek garantáltan 1-esek. Úgy szeretnénk végig jutni a sorozaton, hogy 1-esről 1-esre ugrálunk (kötelező, hogy az első 1-esen dobbantsunk, és az utolsón landoljunk. Adva lévén, hogy maximum mekkorát (hány elemnyit) ugorhatunk, mennyi az a minimális ugrásszám, amellyel át tudunk jutni a bináris sorozaton? Írjunk Pascal vagy C/C++ programot, amely billentyűzetről beolvassa az n értékét ($1 < n < 1001$), az n elemű bináris sorozatot, és a maximális ugrás hosszát, majd kiírja a képernyőre minimális ugrásszámot. Ha az átjutás nem lehetséges, akkor azt írjunk ki, hogy -1.

PÉLDA

Input:

```
11  
1 0 1 1 0 1 0 1 1 0 1  
3
```

Output: 4

```
#include <stdio.h>  
int main() {  
    int i, p, q, n, a[1001], maxugras, lepesszam = 0;  
    freopen("be.txt", "r", stdin);
```

```

scanf("%i", &n);
for ( i = 0 ; i < n ; ++i ){
    scanf("%i", &a[i]);
}
scanf("%i", &maxugras);
p = 0;
while (p < n-1 ){
    ++lepasszam;
    q = p + maxugras;
    if ( q > n-1 ) { break; }
    while ( !a[q] ){ --q; }
    if ( p == q ){
        lepasszam = -1;
        break;
    }
    p = q;
}
printf ("%i", lepasszam );
return 0;
}

```

9. (1 pont) Legyen háromféle pénzérme, amelyek értékei p_1 , p_2 és p_3 ($0 < p_1, p_2, p_3 < 101$). Bármelyikből van bármennyi. Határozzuk meg, hogy minimum hány pénzérmmel fizethető ki egy adott S összeg ($0 < S < 1001$). Írjunk Pascal vagy C/C++ programot,

(a) amely billentyűzetről beolvassa a három pénzérme (p_1 , p_2 , p_3) és az S értékét, majd kiírja a képernyőre a minimális pénzérme számot;

(b) amely billentyűzetről beolvassa az S értékét, majd kiírja a képernyőre a minimális pénzérme számot arra a speciális esetre, ha a pénzérme értékei 2, 3 és 5 euró.

Ha a kifizetés nem lehetséges, akkor azt írjuk a képernyőre, hogy -1.

PÉLDA

Input (a): 1 3 4 6 (magyarázat: $p_1=1, p_2=3, p_3=4, S=6$)

Output (a): 2 (magyarázat: $6=3+3$)

Input (b): 11 (magyarázat: $S=11$)

Output (b): 3 (magyarázat: $11=5+3+3$)

```

(a)
#include <stdio.h>
int main() {
    int i, p1, p2, p3, S, a[1001];
    scanf("%i%i%i%i", &p1, &p2, &p3, &S);
    a[0] = 0;
    for ( i = 1 ; i <= S ; ++i ){
        a[i] = 1001;
        if ( i-p1 >= 0 && a[i-p1]+1 < a[i] ) { a[i] = a[i-p1]+1; }
        if ( i-p2 >= 0 && a[i-p2]+1 < a[i] ) { a[i] = a[i-p2]+1; }
        if ( i-p3 >= 0 && a[i-p3]+1 < a[i] ) { a[i] = a[i-p3]+1; }
    }
    if ( a[S] < 1001 ){ printf ("%i", a[S] ); }
    else { printf ("-1"); }
    return 0;
}

```

```

(b)
#include <stdio.h>
int main() {

```

```
int S;  
scanf("%i", &S);  
if ( S == 1 ) { printf ("-1"); }  
else{  
    if ( S % 5 == 0) { printf ("%i", S/5); }  
    if ( S % 5 == 1) { printf ("%i", S/5 + 1); }  
    if ( S % 5 == 2) { printf ("%i", S/5 + 1); }  
    if ( S % 5 == 3) { printf ("%i", S/5 + 1); }  
    if ( S % 5 == 4) { printf ("%i", S/5 + 2); }  
}  
return 0;  
}
```